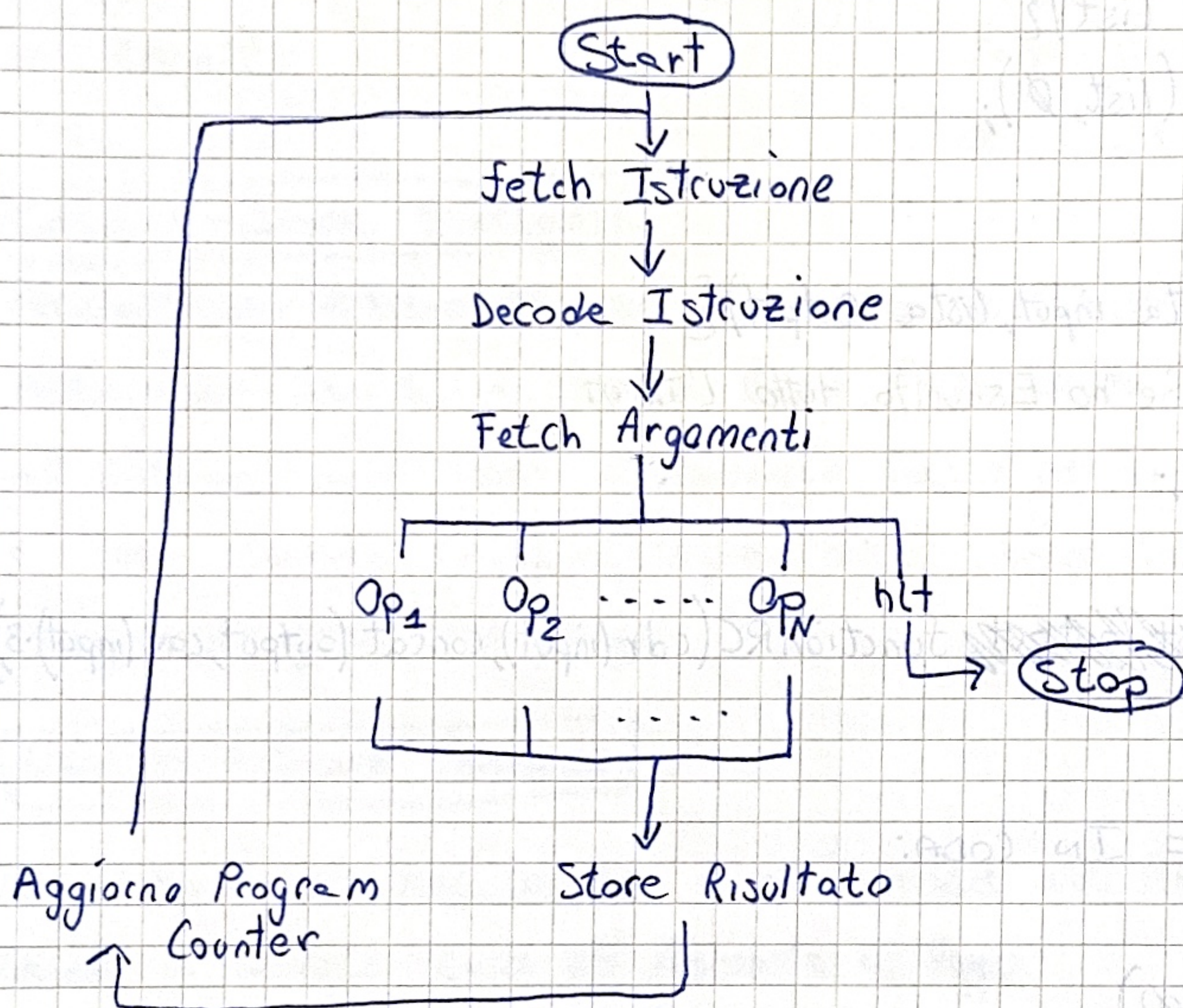


① Un Interprete per L Scritto in L₀ è Quel Programma che realizza Questa funzione: $I^{L_0}: Prog \times Dati \rightarrow dati$

$\downarrow$ INPUT $\rightarrow$ OUTPUT

A differenza del Compilatore l'Interprete ha Le fase DINAMICA e STATICA Unite, ovvero Traduce ed Esegue Simultaneamente



⑤ Una fz. è Ricorsiva se è definita In Termini di Se Stessa, vale a dire che Nel Suo Corpo ha una Chiamata a Se Stessa.

Si dice Ricorsione in coda se la chiamata Ricorsiva Non Viene ulteriormente Elaborata/Manipolata.

Ricorsione in Coda utilizza in Maniera più Efficiente la Memoria costruendo il Risultato chiamate dopo chiamata e restituendolo alla Base della Ricorsione.

ESECUZIONE: (Sole chiamate)

function(2,4,6) $\leftarrow 4 \cdot 3 = 12$
 $\rightarrow$ function(4,6) $\leftarrow 6 \cdot 3 = 18$
 $\rightarrow$ function(6) $\leftarrow$ Nulla
 $\rightarrow$ function($\emptyset$)

Il Risultato è la lista moltiplicata

Per 3 Quindi:

($\emptyset$, 12, 18)

$\rightarrow$ Calcolato Nella Prima Chiamata

```
Lista function (lista list){  
  return functionRC(list,  $\emptyset$ );  
}
```

```
Lista functionRC (lista input, lista Output){  
  IF (len(input) = 0) Se ho Esaurito tutto L'INPUT  
    RETURN output;  
  ELSE {  
    RETURN concat(output, car(input) * 3) functionRC(cdr(input), concat(output, car(input) * 3));  
  }
```

ESECUZIONE RICORSIONE IN CODA:

function(2,4,6)
 $\rightarrow$ functionRC(2,4,6, ($\emptyset$))
 $\rightarrow$ functionRC(4,6, ($\emptyset$), 6)
 $\rightarrow$ functionRC(6, ($\emptyset$), 6, 12)
 $\rightarrow$ functionRC($\emptyset$, ($\emptyset$), 6, 12, 18)
 $\rightarrow$ Ritorna Risultato (6, 12, 18)

④ Devo Avere Nell' Ambiente di definizione e di chiamata due Variabili con Stesso Nome Ma Con Valori diversi in Modo da far Notare la differenza di Approccio.

(*) $\text{int } y = 10;$ ~~$y = 2 + i$~~

→ devo definirla Per Avere Ambiguità Per il Riferimento Non Locale

(**) $y = 2 + i$

→ è già definita, la faccio dipendere da i in Modo Tale da farla Cambiare ad Ogni Iterazione del ciclo

CASO A → SCOPE STATICO!

Ambiente per Riferimenti non Locali è Quello Valido al Momento della definizione Quindi in Questo Caso Pluto essendo definita Nel Blocco più Esterno avrà Come Riferimento locale per la y il Valore 10 e Sarà Sempre Quello, Ovunque Verrà Chiamata. Per Questo $x=10$ Sempre, Ad Ogni Iterazione.

CASO B → SCOPE DINAMICO

Per Riferimenti non LOCALI ci si Riferisce all' Ambiente di Chiamata Quindi in Questo Caso All' Ambiente di Pippo:

- PRIMA ITERAZIONE $\Rightarrow y = 2 + 0$ Quindi $y = 2$ e Quindi Pluto Ritornerà Valore 2 e per Questo $x = 2$ (y dell' Ambiente di Pippo)
- SECONDA ITERAZIONE $\Rightarrow y = 2 + 2$, Quindi $y = 4$. Pluto Ritornerà Val. 4 e $x = 4$ (y dell' Ambiente che ha Effettuato la Chiamata, ovvero Pippo)

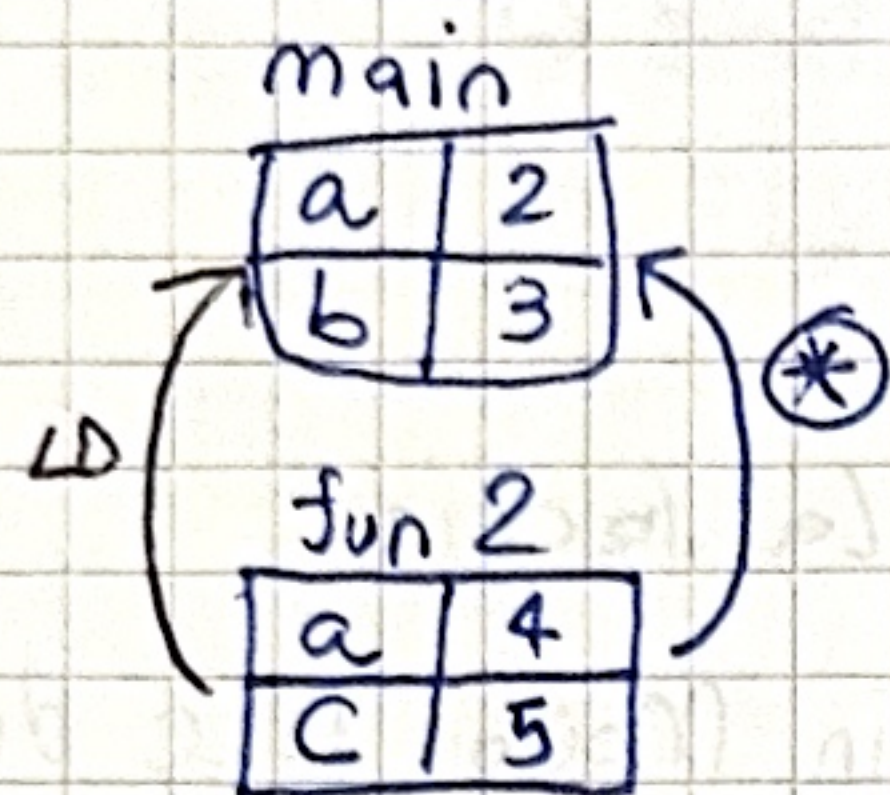
Abbiamo Quindi Visto che Se con lo SCOPE dinamico Sappiamo che il

```
 $\Delta$  int Pluto() {  
    RETURN y;  
}
```

Riferimento non Locale Varia da Esecuzione in EXEC. Allora Otterremo Valori diversi Mentre con lo Statico è SEMPRE Uguali Perché non può Cambiare il luogo di definizione (Sarebbe Programma diverso)

③ Nato che Non Servono Regole di BINDING (Shallow o Deep) perché non Sono State Passate Procedure con Riferimenti non Locali Come Parametri.

SCOPING STATICO:



$$Sd(main) = 0$$

$$Sd(fun) = Sd(fun2) = 1$$

$$Sd(fun3) = 2$$



Sd è la Profondità Statica,
Valore calcolabile a Tempo di Compilazione

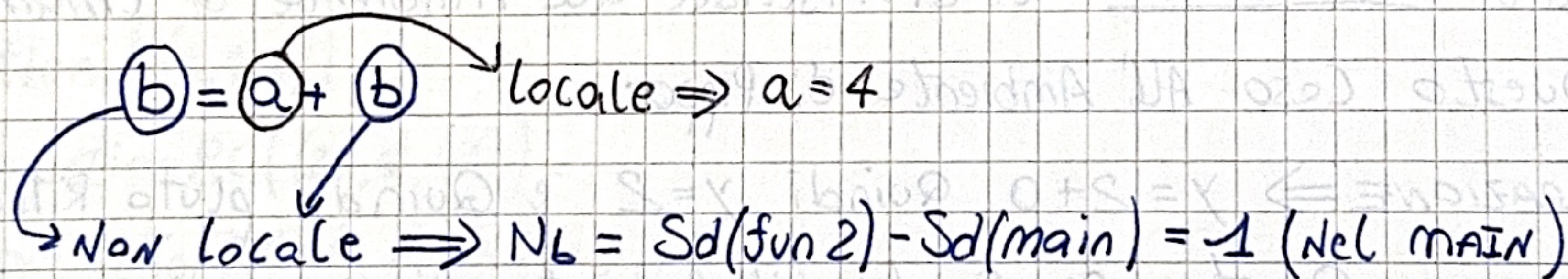
Seq. chiamate $\Rightarrow$ MAIN $\rightarrow$ FUN 2 $\rightarrow$ FUN 3 $\rightarrow$ FUN

IL LINK DINAMICO ^{LD} punta Sempre ALL' RDA Precedente (ovvero Quello che ha Effettuato la chiamata)

⊛ Link Statico = $LS = Sd(\text{Chiamante}) - Sd(\text{Chiamato}) + 1$ e mi Indica chi è il Mio "GENITORE STATICO" ovvero chi Contiene ~~il~~ l'Ambiente dei Miei riferimen-
ti non Locali.

$$LS(fun2) = Sd(main) - Sd(fun2) + 1 = 0 - 1 + 1 = 0 = \text{addr.}(main)$$

Eseguendo fun 2 devo Risolvere un Riferimento non locale:

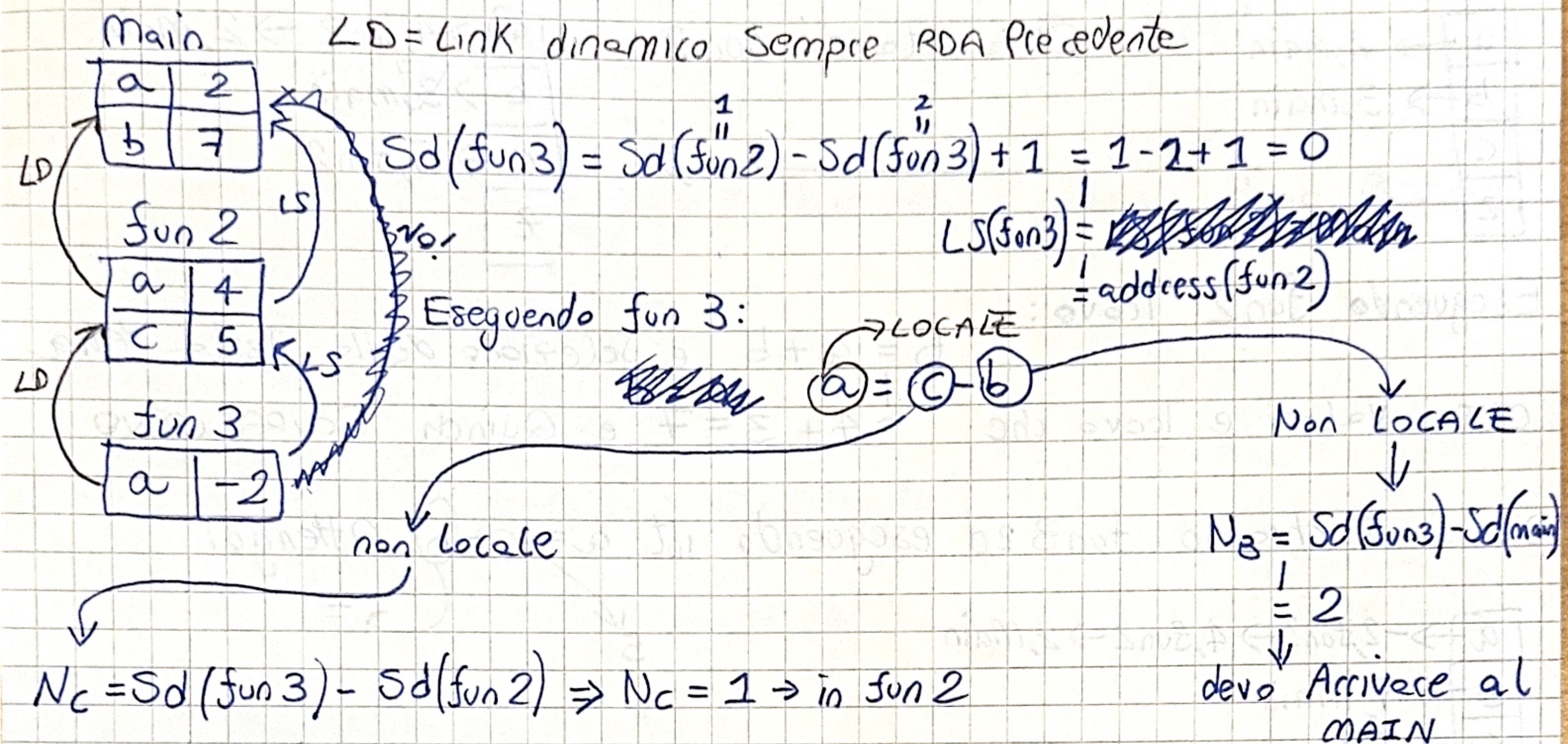


Nb Indica il Numero di Volte che devo Risalire la Catena Statica Per Trovare ~~il~~ l'Ambiente che definisce Quella Variabile, generalmente è così definita:

$$N_b = Sd(\text{chi la usa}) - Sd(\text{Più Vicino che definisce } a)$$

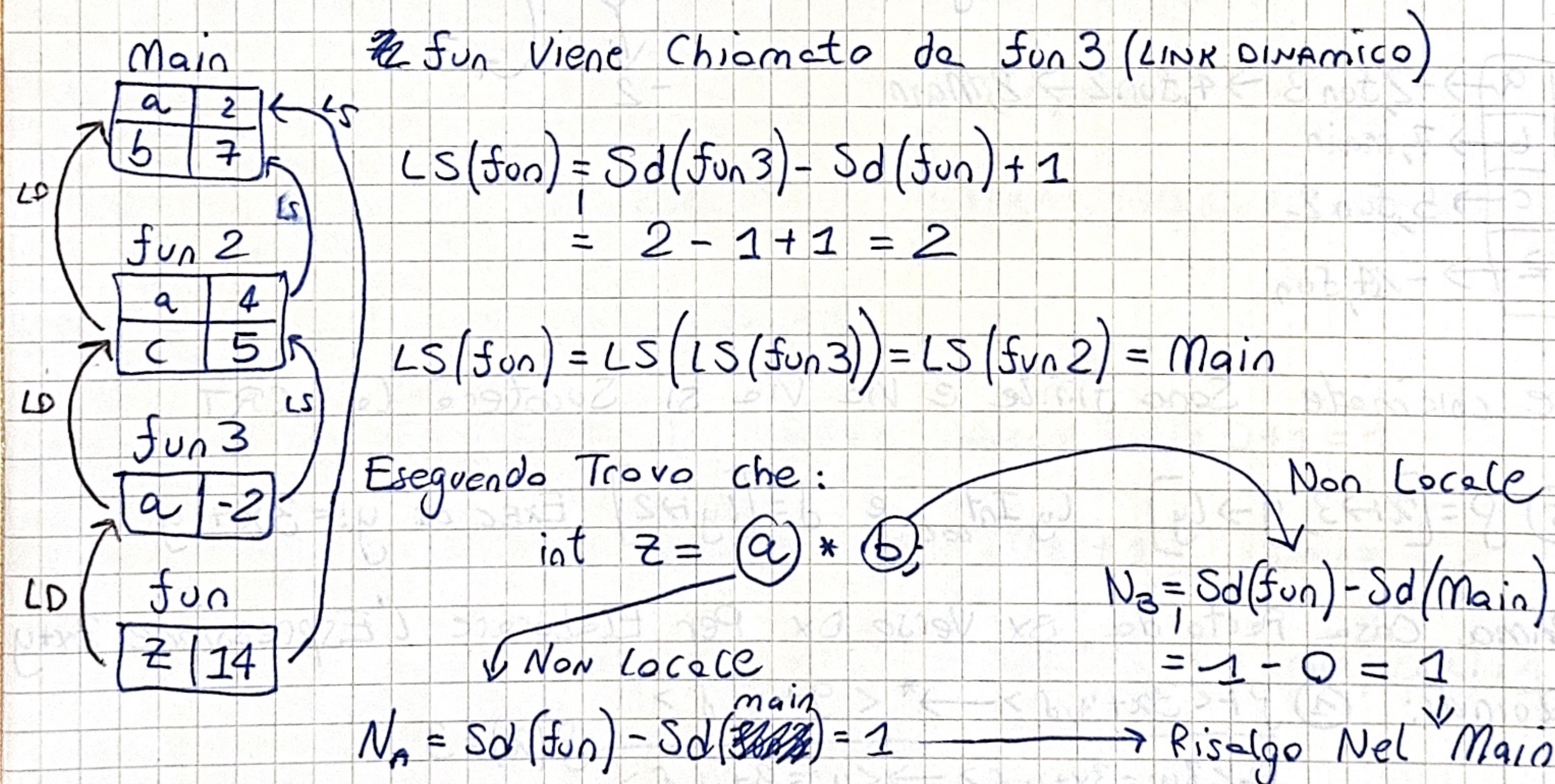
Nel MAIN ho Valore $b = 3$ Quindi, in fun2:

$$b = 4 + 3 \text{ ed Allora } b = 7$$



Quindi ha che $\Rightarrow a = 5 - 7 \Rightarrow a = -2$

Chiamata da fun 3 a fun:

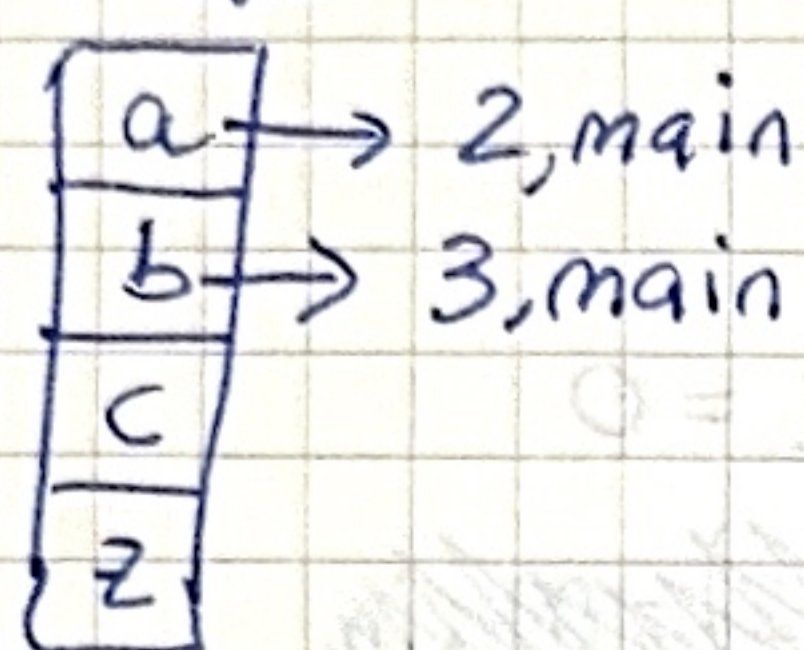


Quindi ho che $\Rightarrow z = 2 * 7 = 14$

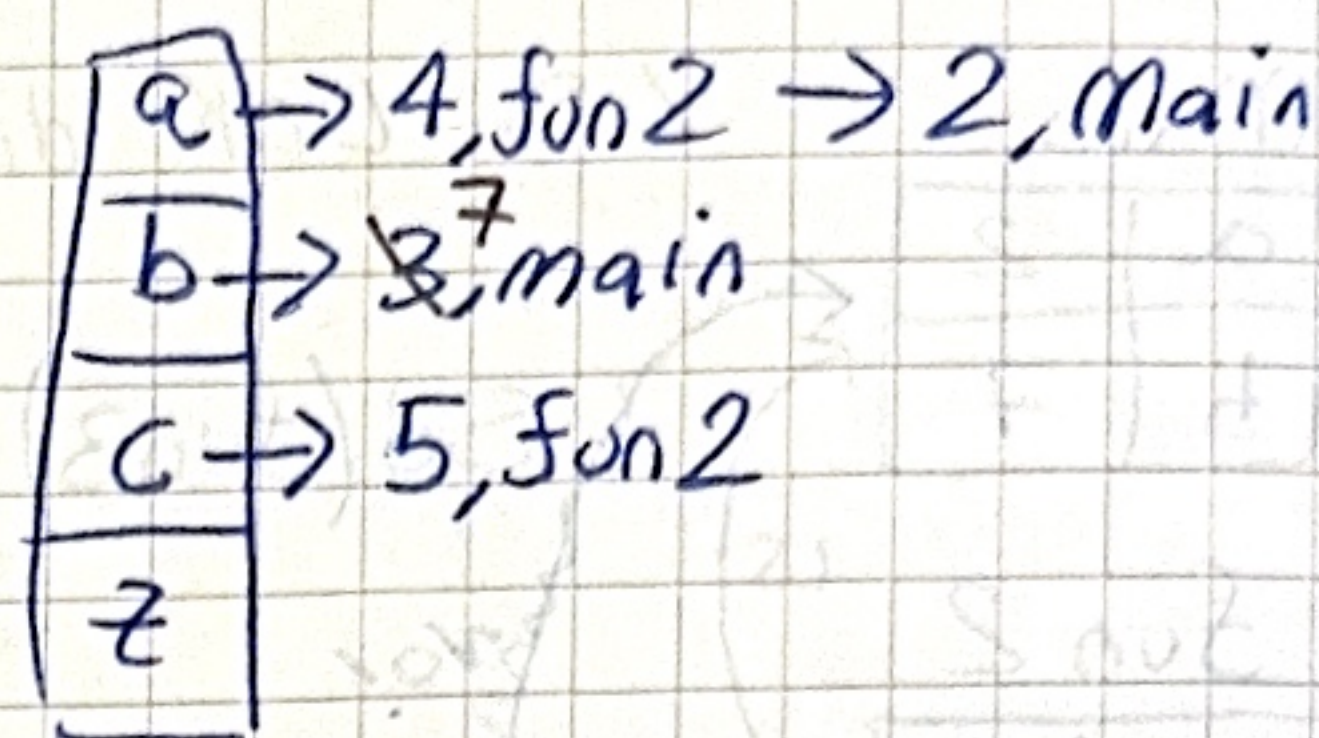
Le chiamate Sono finite e All'uscita delle Procedure L'RDA verrà rimosso e Quindi lo Stack dopo chiusura del MAIN Sarà Vuoto

SCOPING DINAMICO: Uso CRT (Tabelle Centrate dei Riferimenti)

Main



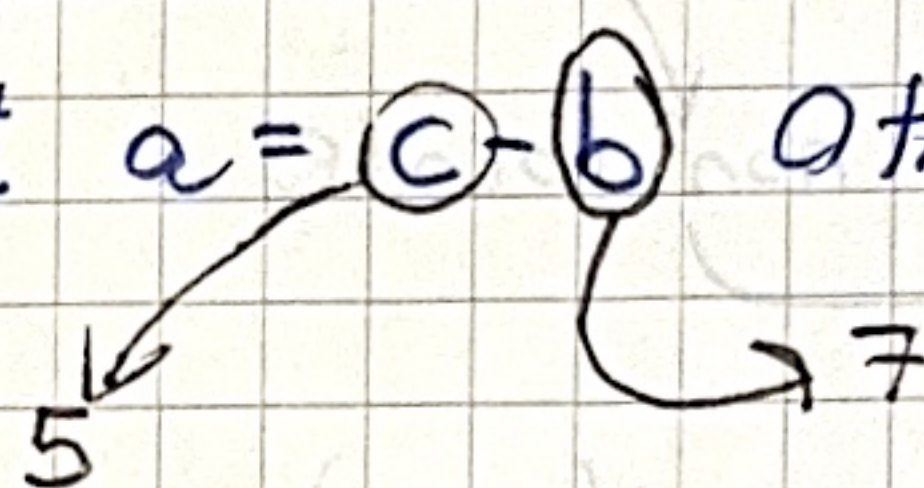
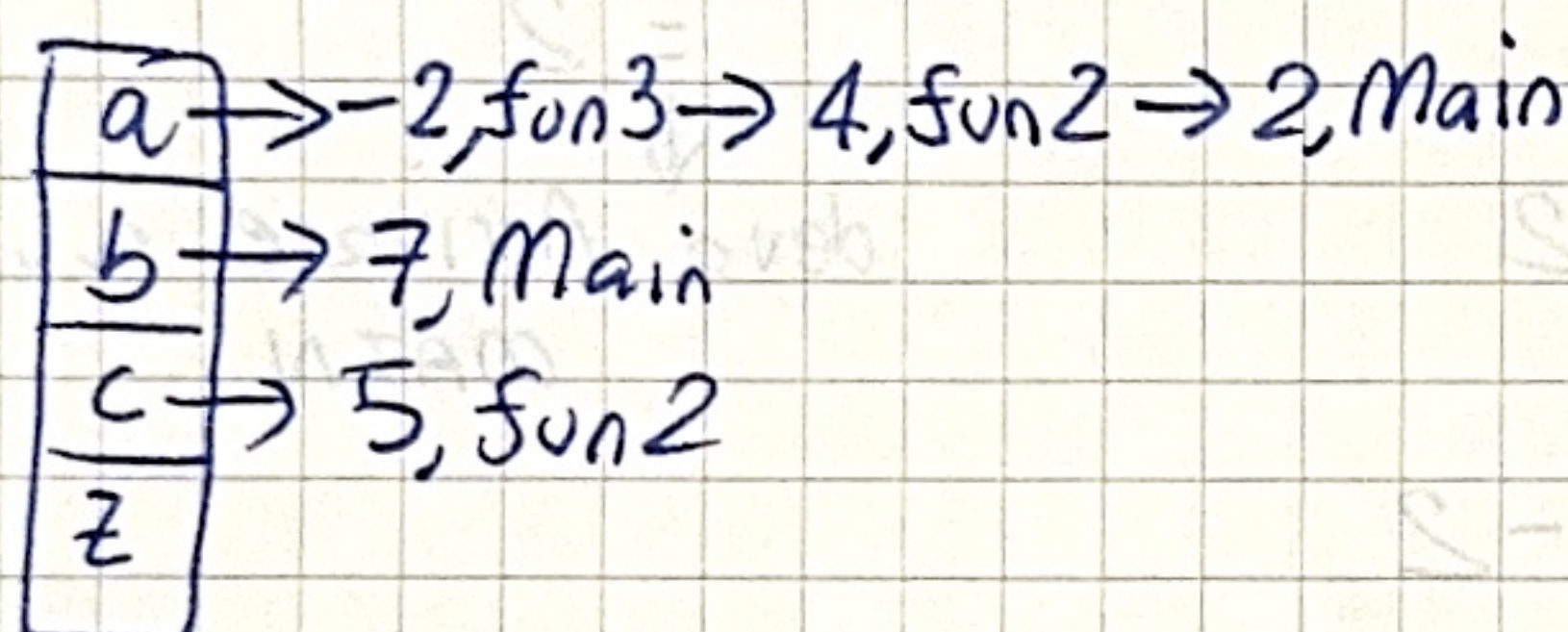
Chiamata di fun 2



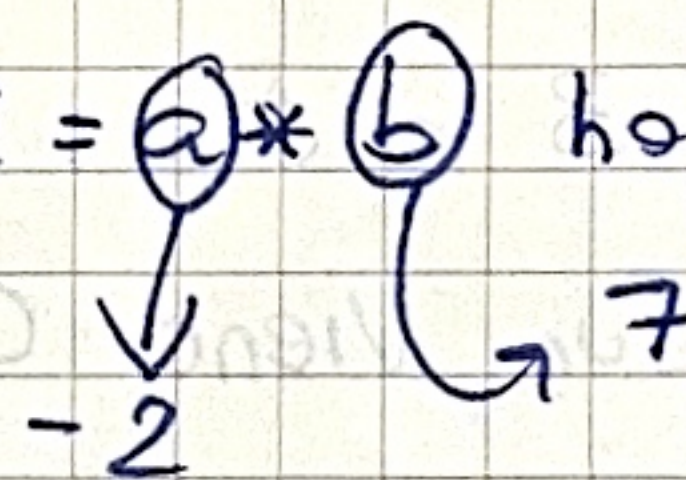
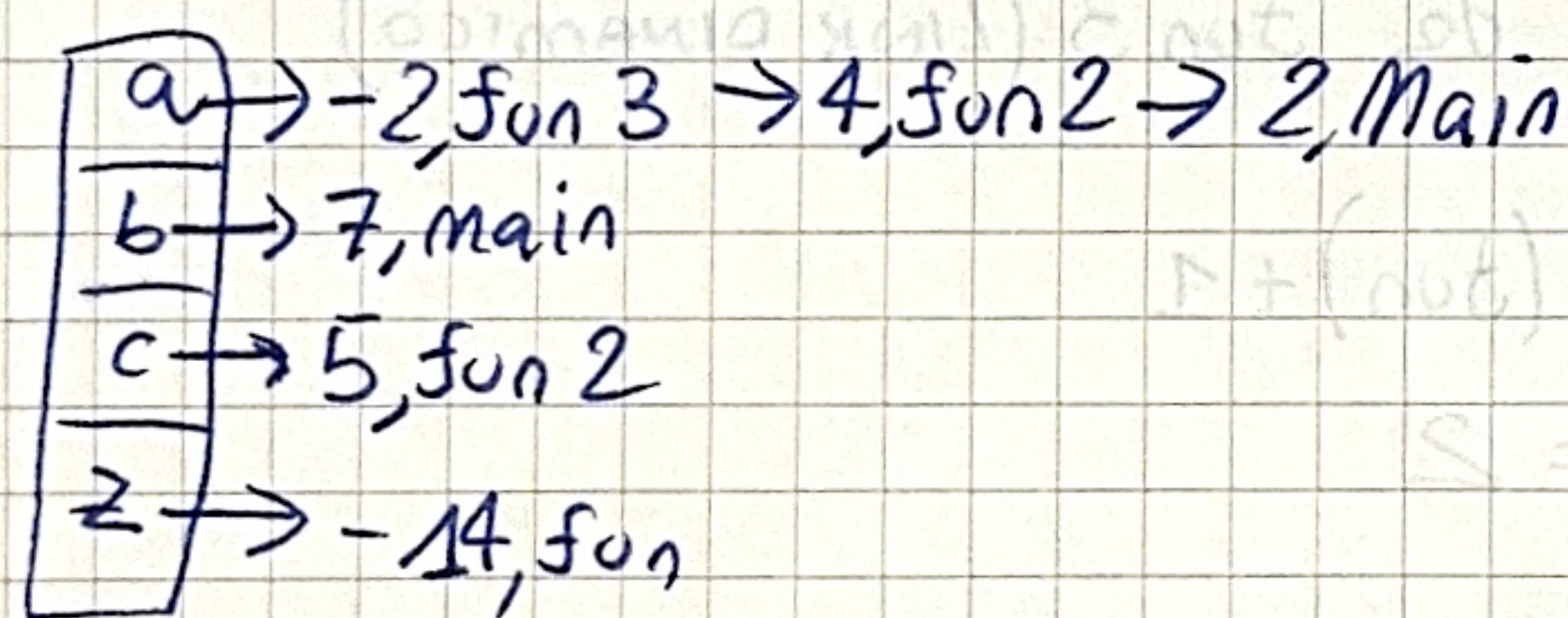
Eseguendo fun2 Trovo:

$b = a + b$ e Selezione della Testa della CRT i Valori e Trovo che $= 4 + 3 = 7$ e Quindi SOVRASCRIVO

Da fun2 chiamo fun3 ed eseguendo $\text{int } a = c - b$ Ottengo:



Chiamando poi fun ed eseguendo $\text{int } z = a * b$ ho che:



Le chiamate Sono finite e Via Via si Svuotano le CRT

⑥ $p = [x \mapsto 3, y \mapsto 4]$ $\hookrightarrow$ Int_{loc} e $d = [y \mapsto 2]$ EXEC DI $y := 3x + y$

Prima Cosa Peto da SX Verso DX Per Elaborare l'Espressione $3x + y$

Quindi: ① $p \vdash \langle 3x + y, d \rangle \rightarrow^* \langle 9 + y, d \rangle$
 $p \vdash \langle y := 3x + y, d \rangle \rightarrow \langle y := 9 + y, d \rangle$ $p(x) = 3$

Ho raccolto l'operazione $3 \cdot 3$ in un UNICO PASSAGGIO, ora devo Valutare y , L'AMBIENTE mi fornisce una locazione che io Sfrutterò Accedendo alla memoria Per osservare il Valore:

$$\frac{p \vdash \langle g+y, \delta \rangle \rightarrow \langle g+2, \delta \rangle}{p \vdash \langle y := g+y, \delta \rangle \rightarrow \langle y := g+2, \delta \rangle} \quad p(y) = l_y \text{ e } \delta[l_y] = 2$$

$$p \vdash \langle y := g+y, \delta \rangle \rightarrow \langle y := g+2, \delta \rangle$$

$$p(y) = l_y \text{ e } \delta[l_y] = 2$$

Devo Solo Ridurmi ad un Valore Terminale K con un ultimo Passo:

$$\frac{p \vdash \langle g+2, \delta \rangle \rightarrow \langle 11, \delta \rangle}{p \vdash \langle y := g+2, \delta \rangle \rightarrow \langle y := 11, \delta \rangle}$$

$$p \vdash \langle y := g+2, \delta \rangle \rightarrow \langle y := 11, \delta \rangle$$

Quindi Termino Con la Memoria Modificata:

$$p \vdash \langle y := 3x+y, \delta \rangle \xrightarrow{*} \delta = [l_y \mapsto 11]$$

② $n \in E$, ~~a~~ $e_1 \wedge e_2 \in E$ se $e_1, e_2 \in E$ e anche $e_1, v e_2 \in E$.

$$\begin{cases} V(n) = 1 \\ V(e_1 \wedge e_2) = V(e_1, v e_2) = V(e_1) + V(e_2) \end{cases}$$

$$V(e_1 \wedge e_2) = V(e_1, v e_2) = V(e_1) + V(e_2)$$

$$\begin{cases} O(n) = 0 \\ O(e_1 \wedge e_2) = O(e_1, v e_2) = O(e_1) + O(e_2) + 1 \end{cases}$$

$$O(e_1 \wedge e_2) = O(e_1, v e_2) = O(e_1) + O(e_2) + 1$$

Da dim. che $\Rightarrow \forall e \in E. v(e) = o(e) + 1$

BASE: Prendo $e = n$ Quindi $V(n) = O(n) + 1$, ovvero $V(n) = 0 + 1 = 1$ ✓

PASSO: Supponendo $e = e_1 \wedge e_2$ ~~devo dim.~~ $V(e) = O(e) + 1$ $\rightarrow$ oppure v è uguale

$$V(e_1, v e_2) = V(e_1 \wedge e_2) = O(e_1 \wedge e_2) + 1 = O(e_1) + O(e_2) + 1$$

Supponendo che $e_1, e_2 \in E$, Allora so che:

$$V(e_1 \wedge e_2) = V(e_1, v e_2) = O(e_1) + O(e_2) + 1 \quad \text{Per IPOTESI INDUTTIVA}$$

$$V(e_1 \wedge e_2) = V(e_1, v e_2) = O(e_1, v e_2) + 1 = O(e_1 \wedge e_2) + 1$$



Corso di Laurea in Informatica

Linguaggi

Esame del 27 Giugno 2025

Si ricorda che ogni risposta va giustificata ed ogni concetto spiegato nel corso che viene citato va definito. Per ogni esercizio si indica tra parentesi il suo valore in 30-esimi.

- ✓ (3) Definire intuitivamente e formalmente (mediante definizione semantica) cosa è un interprete. Descrivere la struttura di un interprete e spiegarne il funzionamento.
- ✓ (3) Si consideri l'insieme E definito induttivamente: $n \in E$, e se $e_1, e_2 \in E$ allora $e_1 \wedge e_2 \in E$, $e_1 \vee e_2 \in E$. Si definiscano sugli elementi di tale insieme le seguenti funzioni: $v(n) = 1$, $v(e_1 \wedge e_2) = v(e_1 \vee e_2) = v(e_1) + v(e_2)$, e $o(n) = 0$, $o(e_1 \wedge e_2) = o(e_1 \vee e_2) = o(e_1) + o(e_2) + 1$. Si dimostri per induzione strutturale che $\forall e \in E. v(e) = o(e) + 1$.
- ✓ (5) Si consideri il programma sulla destra. Si dica cosa viene calcolato dall'assegnamento in caso di scoping statico (mostrando come vengono calcolati i link statici e come vengono risolti riferimenti non locali) e in caso di scoping dinamico (mostrando l'evoluzione della tabella centrale dei riferimenti (CRT) e come vengono risolti riferimenti non locali).

```
{int a = 2; int b = 3; ①
void fun(){
    int z = a * b;}
void fun2(){
    int a = 4; int c = 5;
    void fun3(){
        int a = c - b;
        fun();}
    b = a + b;
    fun3();}
fun2();}
```

- ✓ (6) Si consideri lo schema del seguente codice, nel quale vi sono due buchi indicati rispettivamente con (*) e (**). Si dia il codice da inserire al posto di (*) e (**) in modo tale che:
- ✓ (a) Se il linguaggio usato adotta lo scope statico, le due iterazioni assegnino a x lo stesso valore;
- ✓ (b) Se il linguaggio usato adotta lo scope dinamico, le due iterazioni assegnino a x valori diversi.

```
{ int i=0; int x:=3;
(*)
void pippo(){
    int y;
    (**);
    x:=pluto(); i:=i+2;
}
while(i ≤ 2){pippo();}}
```

Si descrivano dettagliatamente le scelte fatte e si descriva cosa avviene a tempo di esecuzione in entrambi i casi e perché.

- ✓ (5) Definire brevemente il concetto di ricorsione e di ricorsione in coda. Si consideri il programma ricorsivo a destra e si descriva che cosa calcola mostrando anche il calcolo sulla lista (2, 4, 6). Trasformare quindi tale programma in un programma ricorsivo in coda. Mostrare poi il funzionamento dell'algoritmo fornito sulla lista (2, 4, 6).

```
lista function (lista list){
    if (length(list) = 0) then return ();
    else return concat(car(list) * 3, function(cdr(list)));
}
```

NOTA: Data una lista $list$, $length(list)$ restituisce la sua lunghezza, $car(list)$ restituisce il primo elemento, $cdr(list)$ restituisce tutta la lista tranne il primo elemento, $concat$ concatena due liste (se un elemento non è una lista, $concat$ lo converte prima in lista). $()$ è la lista vuota.

- ✓ (5) Siano $\rho = [x \mapsto 3, y \mapsto l_y]$ un ambiente dinamico con l_y locazione di tipo int , e $\sigma = [l_y \mapsto 2]$ una memoria. Descrivere le derivazioni della semantica dinamica per il comando $y := 3x + y$ nell'ambiente ρ a partire dalla memoria σ ($\rho \vdash \langle y := 3x + y, \sigma \rangle \rightarrow \dots$).